

BAHAN_AJAR_Pertemuan10

BAHAN AJAR - PERTEMUAN 10

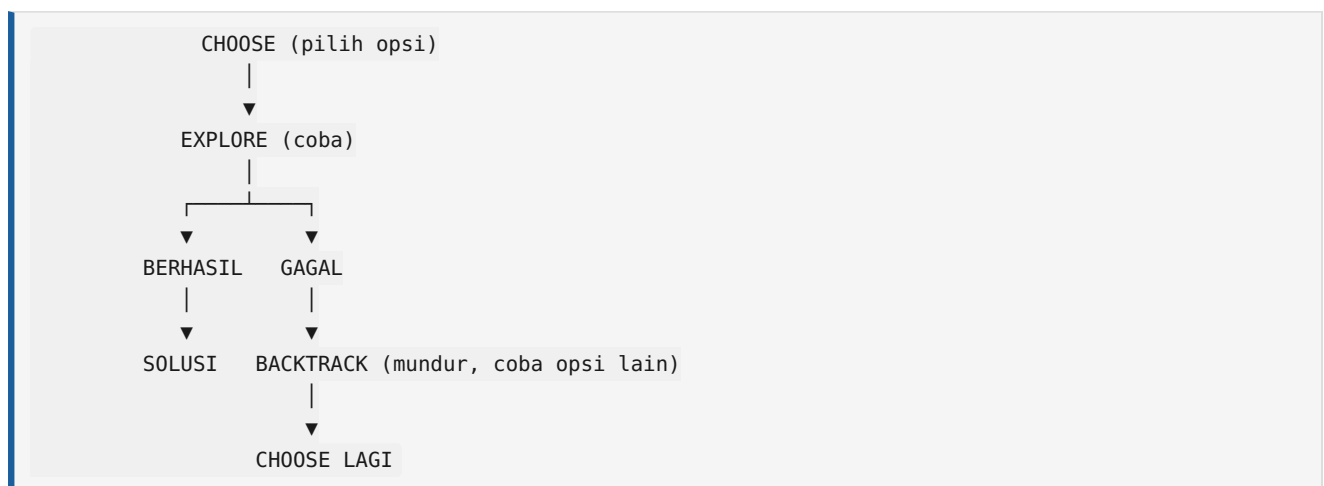
Strategi Algoritmik — Backtracking

TP	BK, AP — Strategi Algoritmik
----	------------------------------

A. APA ITU BACKTRACKING?

Backtracking adalah strategi algoritmik yang mencari solusi dengan mencoba semua kemungkinan secara sistematis. Jika pilihan yang diambil tidak mengarah ke solusi, algoritma akan **mundur** (backtrack) dan mencoba pilihan lain.

5 Langkah Backtracking



Pruning: Hentikan lebih awal jika sudah pasti gagal!

Analogi: Labirin

Situasi	Backtracking
Sampai di persimpangan pilih jalur kiri	Choose
Jalan terus sampai ujung	Explore
Ternyata buntu	Check → gagal
Kembali ke persimpangan	Backtrack
Coba jalur kanan	Choose lagi
“Jalur ini sudah pasti buntu”	Prune

B. MAZE / LABIRIN

Masalah

Cari jalur dari S (Start) ke F (Finish). [X] = rintangan.

	Kolom 0	1	2	3	4
Baris 0	S	[]	[]	[X]	[]
Baris 1	[X]	[]	[X]	[]	[]
Baris 2	[]	[]	[]	[]	[X]
Baris 3	[]	[X]	[X]	[]	[]
Baris 4	[]	[]	[]	[]	[F]

Pohon Pencarian

```

S(0,0)
├── (0,1)
│   ├── (0,2)
│   │   ├── (0,3) [X] ×
│   │   │   └── ← backtrack ke (0,2)
│   │   ├── (1,2) [X] ×
│   │   │   └── ← backtrack ke (0,2)
│   │   └── tidak ada opsi lain
│   │       └── ← backtrack ke (0,1)
│   ├── (1,1)
│   │   ├── (1,0) [X] ×
│   │   ├── (2,1)
│   │   │   ├── (2,0)
│   │   │   ├── (3,1) [X] ×
│   │   │   └── (2,2) → ...
│   │   └── (1,2) [X] ×
│   └── (0,0) sudah dikunjungi ×
└── (1,0) [X] × (pruned)
    
```

Jalur yang Ditemukan

$S(0,0) \rightarrow (0,1) \rightarrow (1,1) \rightarrow (2,1) \rightarrow (2,0) \rightarrow (3,0) \rightarrow (4,0)$
 $\rightarrow (4,1) \rightarrow (4,2) \rightarrow (4,3) \rightarrow (4,4) \rightarrow (3,4) \rightarrow \text{FINISH}$

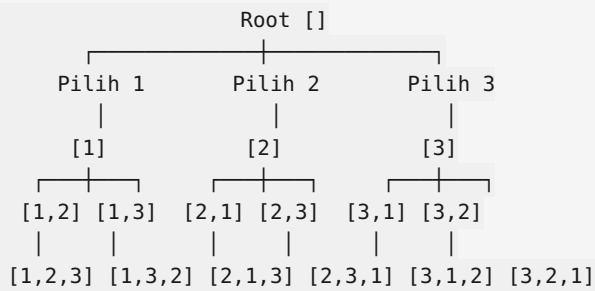
Atau jalur alternatif: $S(0,0) \rightarrow (0,1) \rightarrow (1,1) \rightarrow (2,1) \rightarrow (2,2) \rightarrow (2,3) \rightarrow (1,3) \rightarrow (0,3) \rightarrow (0,4) \dots$ (tapi $(0,3)=[X] \times$)

C. PERMUTASI ANGKA

Masalah

Buat semua kemungkinan susunan (permutasi) dari angka {1, 2, 3}.

Pohon Backtracking Penuh



Jumlah: $3 \times 2 \times 1 = 6$ permutasi ($3!$)

Pseudocode Permutasi

```

PROCEDURE Permutasi(elemen[1..n], terpakai, hasil, semuaHasil)
  IF panjang(hasil) = n THEN
    semuaHasil ← semuaHasil + [hasil] // basis: semua terpakai
    RETURN
  ENDIF

  FOR i ← 1 TO n DO
    IF NOT terpakai[i] THEN
      // 1. CHOOSE
      terpakai[i] ← TRUE
      hasil ← hasil + [elemen[i]]

      // 2. EXPLORE
      Permutasi(elemen, terpakai, hasil, semuaHasil)

      // 3. BACKTRACK (undo)
      hasil ← hasil[1..panjang(hasil)-1]
      terpakai[i] ← FALSE
    ENDIF
  ENDFOR
END
  
```

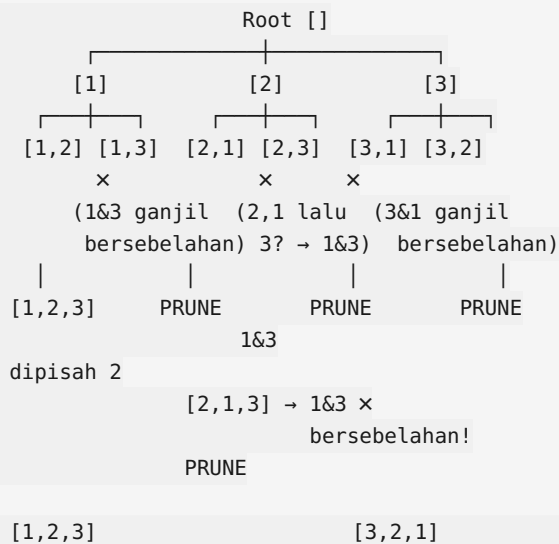
D. PRUNING — MEMOTONG CABANG

Pruning adalah teknik menghentikan pencarian di cabang yang sudah pasti tidak mengarah ke solusi.

Contoh: Permutasi {1, 2, 3} dengan syarat angka ganjil tidak boleh bersebelahan

Ganjil: 1, 3. Genap: 2.

Pohon dengan pruning:



Solusi valid: 1. [1, 2, 3] — 1 dan 3 dipisah angka 2

2. [3, 2, 1] — 3 dan 1 dipisah angka 2

Keuntungan Pruning

Tanpa Pruning	Dengan Pruning
Cek semua 6 permutasi	Hanya 4 cabang yang dieksplorasi
6 pengecekan	4 pengecekan — lebih cepat!
Untuk $n=10$: $10! = 3.628.800$	Jauh lebih sedikit

E. PERBANDINGAN 3 STRATEGI

Aspek	Greedy	D&C	Backtracking
Cara	Ambil terbaik sekarang	Pecah → selesaikan → gabung	Coba → gagal → mundur
Kepastian optimal	Tidak selalu	Ya (untuk masalah tertentu)	Ya (eksplorasi semua)
Kecepatan	Cepat $O(n \log n)$	Sedang $O(n \log n)$	Lambat $O(2^n) / O(n!)$
Contoh	Koin, Knapsack	Binary Search, Merge Sort	Maze, Permutasi, Sudoku
Kapan pakai?	Butuh cepat, optimal tidak kritis	Masalah bisa dipecah	Butuh solusi pasti, n kecil

F. RANGKUMAN

Konsep	Inti
Backtracking	Coba sistematis, mundur jika gagal
Choose	Pilih satu opsi
Explore	Coba opsi tersebut
Backtrack	Kembali jika gagal

Konsep	Inti
Prune	Potong cabang yang pasti gagal
Permutasi	$n!$ kemungkinan susunan
Maze	Cari jalan keluar dengan backtrack
Kompleksitas	$O(n!)$ atau $O(2^n)$ — lambat, hanya untuk n kecil

MGMP Informatika SMAN 6 Cimahi